**Prompt for both AIs**:
Generate a C++ program that meets the following description:
Generate C++ code using objects, not functions, to implement
the following requirements:
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so

on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.

**Analysis:**

**2. Analyze the two GenAI programs. The analysis must include:**
**a. Name of two GenAI programs**
I used ChatGPT and Claude.

**b. How you accessed them (e.g., Google query, CoPilot on my computer)**
I accessed both ChatGPT and Claude on my computer web browser.

**c. Prompt used. (You must use the same prompt for both GenAIs.)**
Prompt used: given above;

**d. Generated code from both GenAIs.**
**Generated code from ChatGPT:**

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

class Sudoku {
private:
   int board[9][9]{};
```

```
int solutionCount = 0;

// Check the row, column, and 3x3 block for a repeated number.
bool canPlace(int row, int col, int number) const {
    for (int i = 0; i < 9; i++) {
        if (i != col && board[row][i] == number) {
            return false;
        }
        if (i != row && board[i][col] == number) {
            return false;
        }
    }

    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if ((r != row || c != col) &&
                board[r][c] == number) {
                return false;
            }
        }
    }
    return true;
}

// Validate the original puzzle.
bool startingBoardIsValid() const {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] != 0 &&
                !canPlace(row, col, board[row][col])) {
                return false;
            }
        }
    }
    return true;
}

// Recursive depth-first search with backtracking.
void solveRecursive() {
```

```
        int bestRow = -1;
        int bestCol = -1;
        int fewestCandidates = 10;

        // Find an empty cell with the fewest possible numbers.
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == 0) {
                    int candidates = 0;

                    for (int number = 1; number <= 9; number++) {
                        if (canPlace(row, col, number)) {
                            candidates++;
                        }
                    }

                    if (candidates == 0) {
                        return; // Dead end: backtrack.
                    }

                    if (candidates < fewestCandidates) {
                        fewestCandidates = candidates;
                        bestRow = row;
                        bestCol = col;
                    }
                }
            }
        }

        // No empty cells means a solution was found.
        if (bestRow == -1) {
            solutionCount++;
            cout << "Solution " << solutionCount << ":\n";
            printBoard();
            cout << '\n';
            return;
        }

        // Try every possible number.
        for (int number = 1; number <= 9; number++) {
            if (canPlace(bestRow, bestCol, number)) {
                board[bestRow][bestCol] = number;
```

```
                solveRecursive();

                // Undo the attempt to find other solutions.
                board[bestRow][bestCol] = 0;
            }
        }
    }

public:
    // Read a Sudoku puzzle from a text file.
    bool loadPuzzle(const string& filename) {
        ifstream input(filename);

        if (!input) {
            return false;
        }

        solutionCount = 0;

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                char value;

                if (!(input >> value)) {
                    return false;
                }

                if (value == '_') {
                    board[row][col] = 0;
                }
                else if (value >= '1' && value <= '9') {
                    board[row][col] = value - '0';
                }
                else {
                    return false;
                }
            }
        }

        return true;
    }
```

```
    // Print the current Sudoku board.
    void printBoard() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == 0) {
                    cout << '_';
                }
                else {
                    cout << board[row][col];
                }

                if (col != 8) {
                    cout << ' ';
                }
            }
            cout << '\n';
        }
    }

    // Start solving and display the result.
    void solve() {
        solutionCount = 0;

        if (startingBoardIsValid()) {
            solveRecursive();
        }

        if (solutionCount == 0) {
            cout << "No solution found\n";
        }
        else {
            cout << "Solutions found: "
                 << solutionCount << '\n';
        }
    }
};

int main() {
    const string filenames[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
```

```
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const string& filename : filenames) {
        Sudoku puzzle;

        cout << "File: " << filename << "\n";

        if (!puzzle.loadPuzzle(filename)) {
            cout << "Unable to read puzzle file.\n\n";
            continue;
        }

        cout << "Original puzzle:\n";
        puzzle.printBoard();
        cout << '\n';

        puzzle.solve();

        cout << "\n------------------------------\n\n";
    }

    return 0;
}
```

**Generated code from Claude:**

```
// Object-oriented brute-force (DFS + backtracking) Sudoku solver.
// Finds ALL solutions for each puzzle file.
//
// Build: g++ -std=c++17 -O2 -o sudoku sudoku.cpp
// Run:   ./sudoku puzzle1.txt puzzle2.txt ... (defaults to puzzle1..5.txt)

#include <array>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
```

```cpp
// ---------------------------------------------------------------------------
// Board: holds the 9x9 grid and knows how to read/print itself and
// answer "what digits may legally go in this cell?".
// ---------------------------------------------------------------------------
class Board {
public:
    static const int SIZE = 9;
    static const int BLOCK = 3;
    static const int EMPTY = 0;

    Board() {
        for (auto &row : cells_) row.fill(EMPTY);
    }

    // Load a puzzle file. '_' (or '0' / '.') is a blank. Returns false on failure.
    bool load(const std::string &filename) {
        std::ifstream in(filename);
        if (!in) return false;
        int r = 0, c = 0;
        char ch;
        while (r < SIZE && in.get(ch)) {
            if (ch >= '1' && ch <= '9') {
                cells_[r][c] = ch - '0';
            } else if (ch == '_' || ch == '0' || ch == '.') {
                cells_[r][c] = EMPTY;
            } else {
                continue;  // whitespace / newlines
            }
            if (++c == SIZE) { c = 0; ++r; }
        }
        return r == SIZE;
    }

    int get(int r, int c) const { return cells_[r][c]; }
    void set(int r, int c, int v) { cells_[r][c] = v; }

    // Find the first empty cell (row-major). Returns false if board is full.
    bool findEmpty(int &row, int &col) const {
        for (int r = 0; r < SIZE; ++r)
            for (int c = 0; c < SIZE; ++c)
                if (cells_[r][c] == EMPTY) { row = r; col = c; return true; }
```

```
        return false;
    }

    // Candidate digits for cell (r,c): not used in its row, column, or block.
    std::vector<int> candidates(int r, int c) const {
        std::array<bool, SIZE + 1> used{};
        for (int i = 0; i < SIZE; ++i) {
            used[cells_[r][i]] = true;
            used[cells_[i][c]] = true;
        }
        int br = (r / BLOCK) * BLOCK, bc = (c / BLOCK) * BLOCK;
        for (int i = 0; i < BLOCK; ++i)
            for (int j = 0; j < BLOCK; ++j)
                used[cells_[br + i][bc + j]] = true;

        std::vector<int> result;
        for (int d = 1; d <= SIZE; ++d)
            if (!used[d]) result.push_back(d);
        return result;
    }

    // Check that the givens don't already violate a rule.
    bool isConsistent() const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                int v = cells_[r][c];
                if (v == EMPTY) continue;
                // Temporarily blank the cell and see if v is still a candidate.
                Board copy = *this;
                copy.cells_[r][c] = EMPTY;
                bool ok = false;
                for (int cand : copy.candidates(r, c))
                    if (cand == v) { ok = true; break; }
                if (!ok) return false;
            }
        }
        return true;
    }

    void print(std::ostream &os) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
```

```
            if (cells_[r][c] == EMPTY) os << "_ ";
            else os << cells_[r][c] << ' ';
        }
        os << '\n';
    }
  }

private:
  std::array<std::array<int, SIZE>, SIZE> cells_;
};

// ---------------------------------------------------------------------------
// Solver: recursive depth-first search with backtracking.
// Collects every solution rather than stopping at the first.
// ---------------------------------------------------------------------------
class Solver {
public:
  explicit Solver(const Board &puzzle) : board_(puzzle) {}

  // Returns all solutions (empty vector => no solution).
  const std::vector<Board> &solveAll() {
      solutions_.clear();
      if (board_.isConsistent()) search();
      return solutions_;
  }

private:
  void search() {
      int r, c;
      if (!board_.findEmpty(r, c)) {      // k == 81: complete solution
          solutions_.push_back(board_);
          return;
      }
      for (int cand : board_.candidates(r, c)) {
          board_.set(r, c, cand);         // try it
          search();                       // recurse with k+1 cells filled
          board_.set(r, c, Board::EMPTY); // backtrack
      }
      // No candidates (or all failed) => fall through, reporting nothing.
  }

  Board board_;
```

```
    std::vector<Board> solutions_;
};

// ---------------------------------------------------------------------------
// PuzzleRunner: loads one puzzle file, solves it, and reports the results.
// ---------------------------------------------------------------------------
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string &filename) : filename_(filename) {}

    void run(std::ostream &os) {
        os << "Puzzle file: " << filename_ << "\n";
        Board puzzle;
        if (!puzzle.load(filename_)) {
            os << "Could not read a valid puzzle from this file.\n\n";
            return;
        }

        os << "Puzzle:\n";
        puzzle.print(os);
        os << '\n';

        Solver solver(puzzle);
        const std::vector<Board> &solutions = solver.solveAll();

        if (solutions.empty()) {
            os << "No solution found\n";
        } else {
            os << "Solutions found: " << solutions.size() << "\n\n";
            for (size_t i = 0; i < solutions.size(); ++i) {
                os << "Solution " << (i + 1) << ":\n";
                solutions[i].print(os);
                os << '\n';
            }
        }
        os << "----------------------------------------\n\n";
    }

private:
    std::string filename_;
};
```

```
int main(int argc, char *argv[]) {
    std::vector<std::string> files;
    for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    if (files.empty())
        for (int i = 1; i <= 5; ++i)
            files.push_back("puzzle" + std::to_string(i) + ".txt");

    for (const std::string &f : files) {
        PuzzleRunner runner(f);
        runner.run(std::cout);
    }
    return 0;
}
```

**e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.**

**Correctness:** Both ChatGPT and Claude successfully solve the Sudoku puzzles using recursive backtracking. They both accurately check rows, columns, and 3x3 boxes before placing numbers. ChatGPT is stricter about inputs, as it rejects bad characters in the input file. Claude, on the other hand, accepts other symbols for empty cells. Overall, they are both good in terms of correctness.

**Execution time:** Both programs use depth-first search. This gives them a worse-case scenario time complexity of O(9^k), if k is the number of empty cells. Both algorithms are slightly different, though. ChatGPT searches for the cell with the fewest possible candidates before choosing a number. This can reduce recursive calls, making the program slightly faster. Claude, however, finds the first cell and tries all candidates. Claude puts in less work by finding the first cell available. However, it risks needing more recursive calls, which would take more time. I think ChatGPT is stronger because it might prevent recursion, which could therefore improve both execution time and space complexity.

**Space Complexity:** Both programs store the inputted Sudoku board and use memory for recursive calls. Claude stores each found solution in a vector before displaying them. ChatGPT prints solutions as they are found. This means Claude requires more memory

for storing found solutions. Therefore, ChatGPT is more space-efficient, especially if a puzzle has multiple solutions.

**Maintainability:** The organization of each program differs slightly. ChatGPT uses a single Sudoku class that handles reading the puzzle, checking candidate numbers, solving the puzzle, and displaying the results. Claude separates these responsibilities into three classes, including Board, Solver, and PuzzleRunner. This makes Claude's program easier to identify and modify individual parts. ChatGPT has all responsibilities packed into one class, which can be overly complex. Claude also has more thorough comments. ChatGPT has very minimal comments and does not describe many comments or lines. Claude also needs more comments. However, there are prologue comments and more effective in-line comments. ChatGPT also has very descriptive variable names compared to Claude, such as bestRow, bestCol, and fewestCandidates. Overall, in terms of maintainability, Claude has stronger organization and commenting.

**f. Select one to use as the basis for this assignment and justify why you picked it.**

For the basis of my assignment, I am going to use the ChatGPT code. Although Claude has slightly stronger organization, ChatGPT has a better foundation for harder puzzles because it looks for squares that have fewer candidates. Further, ChatGPT immediately prints solutions and therefore has better space complexity than Claude. Additionally, ChatGPT has stricter input validation and descriptive variable names, making the code easier to maintain and understand.

**g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.**

**Correctness:** To improve the correctness of the ChatGPT code, I improved file validation. Now, the code ensures the input file has 9 rows and 9 columns and rejects invalid data.

**Execution time:** To improve the execution time, I changed the code so that it stops searching for a better cell once it finds one that can only have one answer. This reduces unnecessary candidate checking.

**Space Complexity:** To improve space complexity, I changed the variable type used for the values in the puzzle. Since the values are always between 1 and 9, an unsigned char can be used. This saves storage compared to using ints.

**Maintainability:** To improve maintainability, I first added comments to every line of code and descriptions of functions. This will make it easier for people to understand the code during maintenance. Next, I added constants for BLOCK_SIZE, BOARD_SIZE, and EMPTY. This reduced the use of hardcoded numbers, like 9 or 3. This process overall helps with reading code and ensuring variables are consistent.